

Individual Sprint Retrospective – Sprint 1 (Taiga Task #12)

During this sprint, I was responsible for implementing the third-party login functionality for the system. I created and committed the `index.html` and `auth.js` files, which handle the user interface and authentication logic respectively. The `index.html` file provides the login interface, while `auth.js` manages the integration with the third-party authentication provider and processes user login requests.

One of the main challenges I encountered was ensuring that the authentication feature was designed in a way that would integrate smoothly with the rest of the system. Since different parts of the application (such as backend APIs and user management) are being developed in parallel, I had to think carefully about how user data would be structured and passed after authentication. Another difficulty was understanding how third-party authentication flows work, particularly handling tokens and ensuring secure login without exposing sensitive information.

If I were to approach this task again, I would spend more time upfront planning the integration points with the backend, especially defining clear interfaces for how authenticated user data should be handled across the system. I would also research best practices for authentication earlier to avoid some trial-and-error during implementation.

This feature fits into the overall system as the entry point for users accessing the platform. The third-party login simplifies the registration and login process, allowing users to securely authenticate without creating new credentials. Once integrated with the backend, it will connect to user profile management, group functionality, and transaction features, ensuring that only authenticated users can access and interact with the system. This makes it a foundational component for both security and user experience in the application.

Individual Sprint Retrospective– Sprint 2 (Taiga User Story #16)

What I Personally Built

I implemented the **Live SA Interest Rates & Savings Projection** feature, which addresses the project requirement: "*Live interest rate display and savings projections*".

Specifically, I built:

1. **A reusable JavaScript module** (`js/interest-rates.js`) that:
 - Displays the current South African repo rate (6.75%) and prime lending rate (10.25%)
 - Calculates compound interest savings projections based on user's monthly contribution and selected time period
 - Provides real-time updates as users adjust input values
 2. **A responsive UI card component** integrated into three dashboard pages:
 - Admin dashboard (`admin-dashboard.html`)
 - Member dashboard (`member-dashboard.html`)
 - Treasurer dashboard (`treasurer-dashboard.html`)
 3. **The compound interest calculation logic** that shows members how their savings could grow over time using the current prime rate.
-

What I Found Difficult

The main challenge was displaying real-time rates from an external South African data source. Initially, I attempted to fetch live data from SARB's API, but encountered CORS issues and inconsistent API documentation. I had to pivot to using a Stellenbosch University's EconData API.

Another difficulty was ensuring the JavaScript loaded correctly across all three dashboards without breaking existing functionality. I initially used `type="module"` which caused issues, then corrected it to a regular script tag after debugging with browser console logs.

What I Would Do Differently

If I were to do this again, I would:

1. **Set up a simple backend proxy** to handle the SARB API requests, avoiding CORS issues entirely while keeping the frontend clean.
 2. **Write unit tests** for the compound interest calculation function before integrating it into the UI, ensuring accuracy across edge cases (zero contributions, negative values, etc.).
 3. **Create a reusable Web Component** for the interest rate card instead of duplicating HTML across three dashboard files, making future updates easier.
 4. **Add loading states and error handling** with user-friendly messages if the rate fetch fails, rather than just console errors.
-

How My Contribution Fits Into the Overall System

This feature sits at the intersection of **financial analytics** and **member engagement**. It directly supports two key project requirements:

- **Requirement: "Live interest rate display and savings projections"** – Members can see current SA rates and project their savings growth, helping them understand the long-term value of consistent contributions.
- **Requirement: "Analytics and financial reporting"** – The projection calculator provides personalized financial insight, which complements the broader analytics dashboard (contribution compliance, payout history).

From a system architecture perspective, this feature:

- Lives in the **frontend presentation layer** (js/ and HTML files)
- Currently uses **static data** with a clear migration path to a live API
- Enhances the **member experience** by adding educational/value-add functionality beyond basic contribution tracking

My commits (`git log --author="emmanuelmanuel328@gmail.com"`) show I created `js/interest-rates.js` and modified the three dashboard HTML files. The Pull Request (#10) was successfully merged into `main` on [insert date].

Git Commit Evidence

To verify my contributions, run:

bash

```
git log --author="emmanuelmanuel1328@gmail.com" --oneline
```

Expected commits:

- [commit hash] Merge pull request #10 from Boitumelo-555/feature/interest-rates
- [commit hash] Add live SA interest rates and savings projection feature

During this sprint, I was responsible for implementing the meetings management functionality for the Stokvel Management Platform. My work focused mainly on creating the meetings.html page and the meetings.js logic connected to user stories TG -37, TG -38, TG-45, and TG-46, as well as tasks TG- 48, TG- 49, TG- 56, and TG- 57. I built features that allow treasurers and admins to schedule meetings, record minutes for past meetings, and delete cancelled or incorrect meetings. I also implemented the member-facing functionality for viewing upcoming meetings and agendas fetched from the database.

One of the main things I contributed was the scheduling form, which allows admins and treasurers to create meetings with details such as the title, date, time, location, and agenda. I also implemented role-based access control so that only authorized users can see and use the scheduling and deletion features. Another important part of my contribution was building the meeting display system that separates upcoming meetings from past meetings and renders them dynamically from database data. I also added the functionality to save minutes for past meetings and reload the interface after updates.

The most difficult part of this sprint was handling asynchronous database operations and ensuring that the UI updated correctly after creating, deleting, or editing meetings. I also had to carefully manage role permissions so that members only had read-only access while admins and treasurers had editing capabilities. Debugging date and time formatting for meetings was another challenge because the scheduled meeting times had to display consistently.

If I could do this sprint again, I would spend more time improving component reusability and reducing inline styling in the generated HTML. I would also improve error handling and add more validation for form inputs. My work fits into the overall system by improving communication and organization within stokvel groups, ensuring that members can stay informed about upcoming meetings while administrators can manage meeting records efficiently.

Individual Sprint Retrospective - Sprint 4

Throughout this project, I learned a great deal about full-stack software development, teamwork, and building features that solve real user problems. My main contribution was implementing the “Record Contributions” functionality for treasurers in `contributions.js` and the related HTML page. This feature allowed treasurers to manually record cash or offline payments on behalf of members, which improved the usability of the stokvel management system for real-world scenarios where not all payments happen online.

One of the things I am most proud of is successfully integrating multiple pieces of functionality into a single feature. I added a dynamic contribution form, group and member selection, automatic amount population, contribution status handling, and table refresh functionality after recording a payment. I also worked on the PayFast integration and payment confirmation flow, which helped me understand how payment systems interact with frontend applications. Seeing the feature work from end to end gave me confidence in my ability to implement practical business requirements.

A major lesson I learned during this project was the importance of planning and structuring code early. As the application became larger, maintaining clean and modular code became more difficult. If I were to start the project again, I would spend more time designing reusable components and separating logic into smaller modules from the beginning. I also realized the importance of handling edge cases, validation, and merge conflicts carefully. During development, I encountered issues with conflicting HTML changes and UI rendering, which taught me the value of version control discipline and frequent communication within a team.

This project also improved my understanding of asynchronous JavaScript, Supabase queries, DOM manipulation, and role-based functionality. Before this project, I had limited experience connecting frontend interfaces to backend data in a real application. Working on this system helped me understand how user roles, payments, database records, and UI updates all connect together in a production-like environment.

Overall, this project helped me grow both technically and professionally. I became more confident debugging problems independently, integrating APIs, and contributing meaningful features to a collaborative software project.